



INSTITUT NATIONAL SUPERIEUR DES
SCIENCES ET TECHNIQUES D'ABECHE

RAPPORT DE PROJET PROFESSIONNELLE ET PERSONNELLE

*3ème Année en Génie Informatique
Option Génie Logiciel*

IA pour la conversation de schéma UML en code fonctionnel

Réalisé par :

Abakar Mahmat Brahim

Encadreur (s) :

Encadrant Professionnel : Dr. MAHAMAT HABIB

Encadrant Pédagogique : M. ABOUBAKAR

Année universitaire : 2025/206

Dédicace

Dédicace

Remerciements

Je tiens à remercier tout d'abord Dieu le tout puissant pour m'avoir donné la force et la santé de mener à bien ce projet.

Mes remerciements s'adressent également à :

- L'administration de l'INSTA pour le cadre d'étude.
- Mes enseignants du département de Génie Logiciel pour la qualité de la formation reçue.
- Mes encadrants, ABOUBAKR et MAHAMAT ALHABIB, pour leurs conseils précieux et leur suivi constant.
- Ma famille et mes amis pour leur soutien moral et matériel tout au long de mon cursus académique.

Résumé

Ce projet porte sur le développement d'une plateforme web intitulée **UML-to-Code**, conçue pour automatiser la transition entre la phase de conception (UML) et la phase de développement d'un logiciel. L'application permet aux développeurs de soumettre des diagrammes de classes sous divers formats (XMI, PNG/JPG, PDF) et de générer instantanément du code source structuré en Python ou PHP, ainsi qu'un serveur Flask complet.

L'innovation majeure de cette solution réside dans l'intégration de l'Intelligence Artificielle générative (via Groq Llama 3.3-70b), permettant une analyse intelligente des images de diagrammes, complétant ainsi l'approche algorithmique classique basée sur le parsing XMI. Les résultats montrent un gain de temps significatif dans la mise en place des structures de base des projets informatiques.

Mots-clés : UML, Génération de code, Intelligence Artificielle, React, Flask, Llama 3.3, Génie Logiciel.

Abstract

This project focuses on the development of a web platform entitled **UML-to-Code**, designed to automate the transition between the design phase (UML) and the software development phase. The application allows developers to submit class diagrams in various formats (XMI, PNG/JPG, PDF) and instantly generate structured source code in Python or PHP, as well as a complete Flask server.

The major innovation of this solution lies in the integration of generative Artificial Intelligence (via Groq Llama 3.3-70b), enabling intelligent analysis of diagram images, thus complementing the traditional algorithmic approach based on XMI parsing. The results show a significant time saving in setting up the basic structures of IT projects.

Keywords : UML, Code Generation, Artificial Intelligence, React, Flask, Llama 3.3, Software Engineering.

Table des matières

Dédicace	II
Remerciements	III
Résumé	IV
Abstract	V
Liste des sigles et acronymes	X
Introduction Générale	1
1 Analyse et Spécification des Besoins	3
1 Contexte et Motivation du projet	3
2 Le Format XMI (XML Metadata Interchange)	3
2.1 Structure d'un fichier XMI	3
3 Étude de l'existant	4
3.1 La valeur ajoutée de UML-to-Code	4
4 Analyse des besoins	5
4.1 Besoins fonctionnels	5
4.2 Besoins non fonctionnels	5
5 Conclusion du chapitre	5
2 Conception de la Solution	6
1 Approche Théorique : L'Ingénierie Dirigée par les Modèles	6
1.1 Le concept du MDD (Model-Driven Development)	6
1.2 L'Architecture MDA (Model-Driven Architecture)	6
2 Stratégies de Transformation	7
2.1 L'Approche Algorithmique (Déterministe)	7
2.2 L'Approche par IA Générative (LLM)	7
3 Architecture Globale du Système	7
3.1 Architecture Client-Serveur	7
4 Modélisation UML de la Solution	8
4.1 Diagramme de Cas d'Utilisation	8

4.2	Diagramme de Séquence (Génération via IA)	8
5	Conclusion du chapitre	8
3	Réalisation et Implémentation	9
1	Méthodologie : Le Cycle de Vie Incrémental	9
2	Environnement Technologique	9
3	Implémentation du Backend et Parsing XML	10
3.1	Le module <code>xml.etree.ElementTree</code>	10
3.2	Orchestration des Générateurs	11
4	Intégration de l'Intelligence Artificielle	11
5	Réalisation de l'Interface Utilisateur (Frontend)	11
5.1	Design System : Glassmorphism	11
5.2	L'intégration de Monaco Editor	11
6	Conclusion du chapitre	12
	Conclusion Générale	13
	Webographie	16

Liste des figures

Liste des tableaux

1.1	Comparaison des outils existants de génération de code	4
3.1	Environnement technologique du projet UML-to-Code	10

Liste des sigles et acronymes

API	Application Programming Interface
CIM	Computation Independent Model
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
IA	Intelligence Artificielle
IDE	Integrated Development Environment
IDM	Ingénierie Dirigée par les Modèles
JSON	JavaScript Object Notation
LLM	Large Language Model
MDA	Model-Driven Architecture
MDD	Model-Driven Development
MOF	Meta Object Facility
OMG	Object Management Group
PFE	Projet de Fin d'Études
PHP	PHP : Hypertext Preprocessor
PIM	Platform Independent Model
PSM	Platform Specific Model
REST	Representational State Transfer
UML	Unified Modeling Language
XMI	XML Metadata Interchange
XML	eXtensible Markup Language

Introduction Générale

Le domaine de l'ingénierie logicielle est en constante évolution, cherchant sans cesse à optimiser les processus de développement pour réduire les délais de livraison tout en maintenant un haut niveau de qualité. Dans le cycle de vie classique du développement logiciel, la phase de conception, souvent modélisée à l'aide du langage UML (Unified Modeling Language), est une étape fondamentale. Elle permet d'établir une architecture claire et de définir les interactions entre les différentes composantes du système avant même d'écrire la première ligne de code.

Cependant, la traduction de ces modèles conceptuels en code source concret reste une tâche laborieuse, répétitive et chronophage. Bien que l'architecture soit définie, le développeur doit manuellement recréer les classes, les attributs, les méthodes et les relations dans le langage cible. Cette étape manuelle est non seulement une perte de temps précieux, mais elle est également sujette à des erreurs humaines qui peuvent introduire des incohérences entre le modèle conçu et l'implémentation finale.

C'est dans ce contexte que s'inscrit notre projet de fin de cycle intitulé **UML-to-Code**. Ce projet a pour ambition d'automatiser cette transition délicate entre la conception et l'implémentation. **UML-to-Code** est une plateforme web intelligente conçue pour transformer automatiquement des diagrammes de classes UML en code source fonctionnel, structuré et prêt à l'emploi.

Notre solution se démarque par une approche hybride innovante :

- **Une approche algorithmique déterministe** : capable d'analyser des fichiers XMI (XML Metadata Interchange) exportés depuis des outils de modélisation standards comme StarUML.
- **Une approche basée sur l'Intelligence Artificielle** : intégrant un modèle de langage avancé (LLM - Llama 3.3 via Groq) pour analyser des diagrammes sous format image (PNG, JPG) ou PDF, offrant ainsi une flexibilité sans précédent aux développeurs qui préfèrent dessiner leurs architectures.

Ce rapport s'attache à décrire de manière exhaustive les différentes phases qui ont jalonné la réalisation de cette plateforme. Le document est structuré de la manière suivante :

- Le **Chapitre 1** dresse le contexte du projet, l'étude de l'existant, ainsi que l'analyse détaillée des besoins fonctionnels et non fonctionnels auxquels l'application doit répondre.
- Le **Chapitre 2** est consacré à la conception de la solution. Il y est abordé l'approche théorique (MDA/MDD), l'architecture logicielle retenue et les différents diagrammes modélisant le comportement du système.
- Le **Chapitre 3** détaille la phase de réalisation et d'implémentation. Nous y présenterons le cycle de vie adopté, les choix technologiques (React, Flask, Groq), ainsi que des extraits de code illustrant les mécanismes clés comme le parsing XML.

Enfin, une **Conclusion Générale** viendra synthétiser les résultats obtenus, les compétences

acquises au cours de ce projet et les perspectives d'évolution envisageables pour la plateforme UML-to-Code.

Chapitre 1

Analyse et Spécification des Besoins

1 Contexte et Motivation du projet

Le présent projet s'inscrit dans le cadre de la formation en Génie Logiciel à l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA). Il vise à concevoir et réaliser une application full-stack répondant à une problématique réelle de productivité logicielle : la transition automatique de la modélisation à l'implémentation.

La modélisation UML est un standard incontournable, mais le temps passé à transcrire des classes UML en code (Python, PHP, etc.) est considérable. La motivation principale de **UML-to-Code** est de fournir un pont transparent, rapide et intelligent entre la conception visuelle et le code métier.

2 Le Format XMI (XML Metadata Interchange)

Pour comprendre le fonctionnement de la génération algorithmique, il est essentiel d'étudier le format XMI. XMI est un standard défini par l'OMG (Object Management Group), l'organisation qui gère également UML et MOF (Meta Object Facility).

Ce format basé sur XML a pour objectif principal de faciliter l'échange de modèles UML entre différents outils de modélisation de l'industrie (tels que StarUML, Enterprise Architect, ou ArgoUML). L'interopérabilité offerte par XMI permet à un modèle créé dans un outil d'être analysé et traité par un autre système de manière fiable.

2.1 Structure d'un fichier XMI

Concrètement, dans un logiciel comme StarUML, l'ensemble des éléments créés (classes, attributs, diagrammes, relations) forme un modèle interne. L'exportation en XMI permet de sérialiser ce modèle en XML, en utilisant des balises standardisées. Voici les éléments majeurs que notre système identifie :

- `<uml:Model>` : Représente le conteneur global du projet.
- `<packagedElement xmi:type="uml:Class">` : Définit l'existence d'une classe.
- `<ownedAttribute>` : Spécifie les propriétés (attributs) d'une classe, incluant leur type et leur visibilité.
- `<ownedOperation>` : Définit les méthodes associées à la classe.

— `<generalization>` : Indique une relation d'héritage entre deux classes.

3 Étude de l'existant

Avant de concevoir notre solution, une étude approfondie des outils existants dans le domaine de la génération de code a été menée. Le tableau 1.1 résume les caractéristiques et les limites de quelques solutions majeures du marché.

Outil	Langages générés	Format d'entrée	Limites principales
Acceleo	Java, Python, PHP	XMI (EMF)	Écosystème très complexe et lourd (Eclipse). Courbe d'apprentissage rude.
AndroMDA	Java EE	XMI	Solution vieillissante, limitée exclusivement à l'écosystème Java.
Plugins StarUML / UML2Code	JS, Java, etc.	.mdj / XMI	Dépendance totale à un logiciel tiers. Interface rigide.
GenMyModel	Java, Python	Interface Web	Modèle commercial payant, code source fermé, dépendance au cloud de l'éditeur.

TABLE 1.1 – Comparaison des outils existants de génération de code

3.1 La valeur ajoutée de UML-to-Code

Face à ces limites, **UML-to-Code** se positionne comme une plateforme web légère, intuitive et universelle. Sa principale valeur ajoutée réside dans :

1. **La double approche** : Contrairement aux outils classiques limités au XMI, notre plateforme accepte des images (PNG, PDF) analysées par l'Intelligence Artificielle.
2. **L'absence d'installation** : Tout se passe dans le navigateur grâce à une interface moderne.
3. **Le multi-langage** : Génération de Python, de PHP et même d'un serveur Flask complet avec routes CRUD prêtes à l'emploi.

4 Analyse des besoins

La phase d'analyse a permis de déterminer les fonctionnalités attendues par le système pour répondre aux exigences des utilisateurs finaux (les développeurs et architectes logiciels).

4.1 Besoins fonctionnels

Le système doit impérativement offrir les fonctionnalités suivantes :

- **Gestion des formats d'entrée** : Uploader et valider des fichiers XMI (exportés depuis StarUML version 2.1), ainsi que des images ou PDF.
- **Sélection des paramètres de sortie** : Permettre le choix du langage cible (Python, PHP) et de l'architecture cible (Code simple ou Serveur Flask).
- **Sélection du moteur de génération** : Laisser l'utilisateur choisir entre le moteur Algorithmique (rapide et hors-ligne) et le moteur IA (intelligent).
- **Édition et Visualisation** : Fournir un éditeur de code intégré (type IDE) permettant de lire, modifier et copier le code généré.
- **Exportation** : Permettre le téléchargement du résultat final sous forme de projet compressé (ZIP) contenant toute l'arborescence.

4.2 Besoins non fonctionnels

Pour garantir une expérience utilisateur de qualité, les critères suivants doivent être respectés :

- **Performance** : La génération algorithmique doit être quasi-instantanée. La génération via IA (faisant appel à un service externe) doit s'effectuer en un temps raisonnable (10 à 15 secondes).
- **Ergonomie et Design** : L'interface doit être visuellement attrayante (thème Glassmorphism), intuitive, et s'adapter aux différents terminaux (Responsivité).
- **Extensibilité** : L'architecture logicielle doit permettre l'ajout futur de nouveaux langages de programmation (Java, C++) sans modification majeure du cœur du système.

5 Conclusion du chapitre

Ce chapitre a permis de définir le cadre précis de notre projet. En comprenant les mécanismes du standard XMI et en analysant les limites des outils existants, nous avons pu formuler des besoins clairs pour UML-to-Code. Le prochain chapitre abordera l'architecture théorique et technique imaginée pour répondre à ces exigences.

Chapitre 2

Conception de la Solution

1 Approche Théorique : L'Ingénierie Dirigée par les Modèles

La conception de notre solution repose sur un socle théorique solide largement reconnu dans le domaine du génie logiciel : l'Ingénierie Dirigée par les Modèles (IDM).

1.1 Le concept du MDD (Model-Driven Development)

Notre projet s'inscrit pleinement dans l'approche **MDD (Model-Driven Development)**. Dans cette méthodologie, le modèle n'est plus considéré comme un simple document de documentation ou un croquis préliminaire, mais comme l'artefact principal et central du processus de développement. L'objectif est de générer tout ou partie du code source du système directement à partir de ce modèle. Cette automatisation garantit que l'implémentation reflète fidèlement la conception, réduisant ainsi la fracture entre l'architecte et le développeur.

1.2 L'Architecture MDA (Model-Driven Architecture)

Pour concrétiser le MDD, nous nous appuyons sur la **MDA (Model-Driven Architecture)**, un cadre conceptuel standardisé par l'OMG. La MDA prône la séparation stricte entre les spécifications fonctionnelles d'un système et son implémentation technologique. Elle distingue trois niveaux d'abstraction majeurs :

- **CIM (Computation Independent Model)** : Le modèle indépendant des calculs. Il représente la vue métier du système, les exigences des utilisateurs, sans aucune considération technologique.
- **PIM (Platform Independent Model)** : Le modèle indépendant de la plateforme. Il décrit la structure et le comportement du système (dans notre cas, le diagramme de classes UML fourni en entrée par l'utilisateur).
- **PSM (Platform Specific Model)** : Le modèle spécifique à la plateforme. Il représente le code source ciblé (Python, PHP, ou Flask) généré par notre application.

L'application **UML-to-Code** agit donc comme le moteur de transformation automatisé (Transformation Engine) permettant de passer du niveau **PIM** au niveau **PSM**.

2 Stratégies de Transformation

Pour réaliser cette transformation PIM vers PSM, notre plateforme innove en proposant deux approches distinctes et complémentaires.

2.1 L'Approche Algorithmique (Déterministe)

Cette méthode est basée sur des règles de transformation strictes et codées en dur. Le système parcourt le fichier XMI et applique des équivalences :

- Un nœud `uml:Class` déclenche la création d'une structure `class NomClasse:` en Python.
- Un nœud `ownedAttribute` avec `visibility="private"` génère des variables encapsulées (`__attribut` en Python).

Avantages : Cette approche est extrêmement rapide, fiable, ne nécessite aucune connexion internet, et donne un résultat 100% prédictible.

2.2 L'Approche par IA Générative (LLM)

Pour contourner les limites du parsing algorithmique (qui exige un fichier XMI parfaitement formaté), nous intégrons une approche sémantique. Le modèle XMI, ou directement une capture d'écran du diagramme, est envoyé à un Large Language Model (Llama 3.3 via l'API Groq) accompagné d'un prompt métier très précis. L'IA analyse visuellement ou textuellement les entités et génère le code. *Avantages :* Une flexibilité immense permettant de comprendre des croquis à main levée, des PDF, et de générer du code métier complexe.

3 Architecture Globale du Système

Pour supporter ces deux approches, nous avons opté pour une architecture logicielle moderne, séparant clairement les responsabilités.

3.1 Architecture Client-Serveur

Le système est divisé en trois tiers principaux :

- **Le Frontend (Client)** : Développé en React.js avec le bundler Vite. Il offre une interface riche ("Single Page Application"), gère les interactions utilisateur, la prévisualisation des fichiers et intègre un éditeur de code avancé (Monaco Editor).
- **Le Backend (Serveur API)** : Développé en Python avec le micro-framework Flask. Il reçoit les fichiers du frontend, orchestre le parsing XML, exécute les algorithmes de génération, prépare les fichiers ZIP, et gère la communication avec les services externes.

- **Le Service IA Externe** : L'API cloud de Groq qui exécute le modèle open-source Llama 3.3 pour les tâches de vision et de génération de texte.

4 Modélisation UML de la Solution

4.1 Diagramme de Cas d'Utilisation

Les acteurs interagissant avec le système sont le **Développeur** (utilisateur final) et le **Système Externe** (API Groq). Les cas d'utilisation principaux pour le développeur incluent :

- Uploader un modèle (XMI, PNG, PDF).
- Configurer la génération (choix du langage et du mode).
- Lancer la génération de code.
- Consulter l'historique des générations.
- Exporter le projet généré (Format texte ou ZIP).

4.2 Diagramme de Séquence (Génération via IA)

Afin de comprendre la cinématique d'une requête complexe, voici les étapes d'une génération utilisant l'IA :

1. **Développeur** → **Frontend** : Soumet une image de diagramme et sélectionne "Génération Python via IA".
2. **Frontend** → **Backend Flask** : Envoi d'une requête POST contenant l'image et les paramètres.
3. **Backend Flask** → **API Groq** : Formation d'un prompt d'instruction couplé à l'image encodée en base64. L'appel réseau est initié.
4. **API Groq** → **Backend Flask** : Après un traitement d'environ 10 à 15 secondes, l'IA retourne le code généré sous forme de texte brut.
5. **Backend Flask** → **Frontend** : Le texte est formaté, encapsulé dans un objet JSON, et renvoyé au client.
6. **Frontend** → **Développeur** : Affichage du code dans le Monaco Editor avec coloration syntaxique.

5 Conclusion du chapitre

L'architecture conçue pour UML-to-Code repose sur les principes de la MDA, traduits par une implémentation client-serveur moderne. La dualité algorithme/IA permet de couvrir un spectre très large de cas d'usage, allant de l'export d'entreprise strict à l'esquisse rapide. Le chapitre suivant s'attachera à détailler les choix techniques et la réalisation concrète de cette conception.

Chapitre 3

Réalisation et Implémentation

1 Méthodologie : Le Cycle de Vie Incrémental

Pour mener à bien le développement de la plateforme **UML-to-Code**, nous avons adopté une méthodologie basée sur un **cycle de vie incrémental**. Contrairement à un modèle classique en cascade ("Waterfall") où toutes les spécifications sont figées avant le développement, l'approche incrémentale consiste à découper le projet en sous-ensembles fonctionnels (les incréments) qui sont développés, testés et intégrés progressivement.

Ce choix méthodologique a apporté plusieurs bénéfices majeurs :

- **Livraison rapide d'un noyau fonctionnel** : Dès les premières semaines, l'interface utilisateur de base a pu être déployée pour validation.
- **Sécurisation technique** : Chaque module sensible a été validé indépendamment. Par exemple, le moteur de parsing XML a été entièrement testé en ligne de commande Python avant même d'être relié au serveur Web Flask.
- **Intégration continue** : Les fonctionnalités avancées telles que l'historique de génération, le mode sombre, ou la connexion à l'IA ont été greffées sur une architecture déjà stable, minimisant le risque de régressions.

2 Environnement Technologique

Afin de garantir la performance, la maintenabilité et l'évolutivité de l'application, l'environnement technologique résumé dans le tableau 3.1 a été sélectionné :

Couche	Technologie	Rôle dans l'application
Backend	Python 3.10+ & Flask	Serveur d'API REST léger et manipulation de données XML.
Frontend	React.js (ES6+) & Vite	Création d'une interface utilisateur dynamique, réactive et performante (SPA).
Intelligence Artificielle	API Groq (Llama 3.3)	Interprétation sémantique de texte et d'images pour la génération de code.
Outils d'Édition	Monaco Editor	Fournir un éditeur de code avancé directement dans le navigateur.
Gestion de version	Git & GitHub	Suivi des modifications et sauvegarde du code source.

TABLE 3.1 – Environnement technologique du projet UML-to-Code

3 Implémentation du Backend et Parsing XML

Le cœur logique du système réside dans le serveur Flask. Son rôle ne se limite pas à faire le pont entre le client et l'IA ; il abrite le moteur algorithmique de génération de code.

3.1 Le module `xml.etree.ElementTree`

L'extraction des données depuis un fichier StarUML (format XMI) repose sur la bibliothèque standard Python `xml.etree.ElementTree`. Cette bibliothèque parcourt le fichier texte XML et le charge en mémoire sous forme d'un arbre d'objets manipulables.

Le processus d'analyse (parsing) suit l'algorithme suivant :

1. **Initialisation** : Le fichier est chargé et les espaces de noms (namespaces) requis par la norme XMI sont définis pour permettre la recherche de balises spécifiques.
2. **Extraction des Classes** : Le script recherche toutes les occurrences de la balise `<packagedElement xmi:type="uml:Class">`. Chaque occurrence instancie un objet Python interne représentant la classe UML.
3. **Extraction des Propriétés** : Pour chaque classe trouvée, une sous-recherche est lancée pour trouver les `<ownedAttribute>` (les variables) et les `<ownedOperation>` (les méthodes). L'attribut XML `visibility` (public, private) est lu pour adapter la syntaxe dans le langage cible (ex : le double underscore `__` en Python pour le private).
4. **Gestion des Relations** : Les balises `<generalization>` sont analysées pour identifier la classe mère d'une classe fille, afin d'écrire correctement la syntaxe d'héritage.

3.2 Orchestration des Générateurs

Une fois l'arbre UML abstrait construit en mémoire, le système fait appel à des classes de type "Générateur" (ex : `python_generator.py`, `php_generator.py`). Ces classes appliquent des *templates* textuels pour produire le code source final. Un module spécialisé, `zip_generator.py`, prend ensuite ces chaînes de caractères, crée des fichiers physiques en mémoire, et les package dans une archive `.zip` prête au téléchargement.

4 Intégration de l'Intelligence Artificielle

Le module `gemma_client.py` est responsable de la communication avec l'API Groq, fournissant l'accès au modèle **Llama 3.3 (70 milliards de paramètres)**. Lorsqu'un utilisateur soumet une image de diagramme, le backend Python encode cette image en Base64 et formule une requête HTTP sécurisée contenant un prompt très strict (Prompt Engineering). Ce prompt ordonne au LLM d'analyser l'image en tant que diagramme UML et de retourner *exclusivement* le code source (sans texte explicatif superflu), assurant ainsi que le résultat puisse être injecté directement dans l'éditeur du client.

5 Réalisation de l'Interface Utilisateur (Frontend)

Le frontend React a été conçu avec une exigence "Premium", visant à offrir l'expérience d'un véritable IDE dans le navigateur.

5.1 Design System : Glassmorphism

L'interface adopte le style visuel *Glassmorphism*, caractérisé par des fonds translucides, des effets de flou d'arrière-plan (`backdrop-blur`) et des bordures subtiles. Ce design met en valeur le contenu principal tout en offrant un aspect moderne et épuré.

5.2 L'intégration de Monaco Editor

L'un des défis majeurs a été l'intégration de **Monaco Editor**, le moteur de coloration syntaxique et d'édition de texte qui propulse Microsoft Visual Studio Code. Grâce à lui, l'utilisateur bénéficie d'une lecture de code confortable (thèmes clair/sombre, numérotation des lignes, mini-map) directement après la génération. L'application conserve également un **historique local** (via `localStorage`) des 10 dernières générations, permettant au développeur de retrouver facilement ses travaux précédents.

6 Conclusion du chapitre

La phase de réalisation a permis de concrétiser les concepts théoriques élaborés précédemment. Le choix d'outils performants (Flask, React) et l'intégration de technologies de pointe (LLM) ont abouti à une application stable, rapide et ergonomique. Les tests effectués valident la précision à 100% de l'approche algorithmique sur le format XMI, et confirment l'impressionnante capacité d'adaptation de l'approche IA face à des diagrammes variés.

Conclusion Générale

La conception et la réalisation de la plateforme **UML-to-Code** ont constitué une expérience hautement enrichissante, marquant l'aboutissement de notre formation en Génie Logiciel à l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA). Ce projet est né d'un constat simple mais critique dans l'industrie du logiciel : la nécessité de réduire le fossé temporel et technique entre la modélisation architecturale et l'implémentation concrète du code source.

Tout au long de ce document, nous avons démontré comment l'approche du Développement Dirigé par les Modèles (MDD) pouvait être matérialisée par un outil moderne et accessible. En partant de l'analyse des besoins jusqu'au déploiement de l'architecture client-serveur, le projet a suivi un cycle de vie incrémental rigoureux, permettant de valider chaque composant technologique.

Les résultats obtenus sont très satisfaisants et répondent intégralement aux objectifs fixés. La plateforme réussit le pari d'offrir :

- Une robustesse absolue via son moteur de parsing XML (`xml.etree`) capable d'extraire la sémantique précise des fichiers XML normalisés par l'OMG.
- Une flexibilité sans précédent grâce à l'intégration de l'Intelligence Artificielle (Llama 3.3), qui brise les barrières des formats d'exportation propriétaires en permettant la génération de code à partir de simples images ou PDF de diagrammes.
- Un confort d'utilisation professionnel, matérialisé par une interface "Glassmorphism" épurée intégrant l'éditeur de code Monaco, véritable standard de l'industrie.

Sur le plan personnel, ce projet de fin d'études a été un formidable terrain d'application pour consolider mes acquis. Il a exigé une maîtrise complète de la pile technologique "Full-Stack" (React.js, Flask, Python), une compréhension profonde des standards d'architecture logicielle (UML, XML/XMI), ainsi qu'une capacité à appréhender et intégrer les technologies émergentes (LLM via API).

Perspectives d'évolution

Bien que fonctionnelle et aboutie, l'application **UML-to-Code** constitue une fondation sur laquelle de nombreuses évolutions peuvent être envisagées pour en faire un outil de qualité industrielle :

- **Extension sémantique** : Prendre en charge d'autres types de diagrammes UML (Diagrammes de Séquence pour générer le corps des fonctions, Diagrammes d'États-Transitions pour générer des machines à états).
- **Diversification linguistique** : Ajouter des générateurs pour d'autres langages fortement typés tels que Java, C++, ou TypeScript, ainsi que la prise en charge de frameworks ORM

(comme SQLAlchemy ou Doctrine).

- **Intégration au flux de travail (CI/CD)** : Développer la solution sous forme d'extension native pour des IDE (Visual Studio Code, IntelliJ) afin que la génération se fasse de manière transparente durant le développement.

En définitive, **UML-to-Code** illustre parfaitement comment l'alliance entre les méthodes classiques d'ingénierie logicielle et les avancées fulgurantes de l'intelligence artificielle peut redéfinir et optimiser le métier de développeur pour les années à venir.

Bibliographie

Webographie

Sites web consultés

1. **PostgreSQL Documentation**

<https://www.postgresql.org/docs/>

Consulté le : **8 mars 2025**

Description : Documentation officielle et complète de PostgreSQL.

2. **Docker Documentation**

<https://docs.docker.com/>

Consulté le : **19 mars 2025**

Description : Guide officiel pour la containerisation, les images et les déploiements.

3. **Kubernetes Documentation**

<https://kubernetes.io/docs/>

Consulté le : **5 mars 2025**

Description : Documentation et meilleures pratiques pour l'orchestration de conteneurs.

4. **Apache NiFi Documentation**

<https://nifi.apache.org/documentation/>

Consulté le : **13 mars 2025**

Description : Guides et référence de NiFi, outil d'orchestration de flux de données.

5. **GitHub Actions Documentation**

<https://docs.github.com/en/actions>

Consulté le : **22 mars 2025**

Description : Documentation officielle sur l'automatisation CI/CD via GitHub Actions.

6. **DevOps Best Practices (Azure DevOps)**

<https://docs.microsoft.com/azure/devops/>

Consulté le : **10 mars 2025**

Description : Guide Microsoft pour les pratiques DevOps incluant CI/CD.

7. **ChatGPT (OpenAI)**

<https://chatgpt.com/>

Consulté le : **3 mars 2025**

Description : Assistant IA conversationnel basé sur GPT, pour la rédaction, la recherche et le codage.

8. **Claude (Anthropic)**

<https://claude.ai/>

Consulté le : **3 mars 2025**

Description : Assistant IA alternatif axé sur la sécurité, la précision et la productivité.

9. **GitHub Copilot**

<https://github.com/features/copilot>

Consulté le : **3 mars 2025**

Description : Assistant IA de programmation pour l'autocomplétion et la génération de code.

10. **Metabase Documentation**

<https://www.metabase.com/docs/>

Consulté le : **12 mars 2025**

Description : Documentation officielle de Metabase pour la visualisation et l'analyse de données.

11. **Apache Spark Documentation**

<https://spark.apache.org/docs/latest/>

Consulté le : **15 mars 2025**

Description : Référence technique pour le traitement distribué et l'analytique big data.

12. **OWASP Top Ten**

<https://owasp.org/www-project-top-ten/>

Consulté le : **21 mars 2025**

Description : Guide des risques majeurs de sécurité applicative, utile pour la sécurisation des APIs et des plateformes BI.

13. **Data Engineering Podcast**

<https://www.dataengineeringpodcast.com/>

Consulté le : **18 mars 2025**

Description : Podcast sur les architectures, outils et retours d'expérience en data engineering.

14. **Allianz Group**

<https://www.allianz.com/en.html>

Consulté le : **2 mars 2025**

Description : Site institutionnel du groupe Allianz, pour la compréhension du contexte métier et des enjeux sectoriels.

Tutoriels et articles techniques

1. **Building Scalable Data Pipelines**

<https://medium.com/data-engineering>

Consulté le : **14 mars 2025**

Description : Articles techniques sur la conception de pipelines de données évolutifs.

2. **Start Data Engineering (blog)**
<https://www.startdataengineering.com/>
Consulté le : **25 mars 2025**
Description : Blog spécialisé sur les pratiques, outils et architectures en data engineering.
3. **Kubernetes Best Practices**
<https://cloud.google.com/kubernetes-engine/docs/best-practices>
Consulté le : **7 mars 2025**
Description : Bonnes pratiques pour la mise en œuvre de Kubernetes sur Google Cloud.
4. **Database Design Patterns (PostgreSQL)**
<https://www.postgresql.org/docs/current/ddl.html>
Consulté le : **17 mars 2025**
Description : Patterns et bonnes pratiques pour la conception de schémas relationnels.
5. **API Design Guidelines (REST)**
<https://restfulapi.net/>
Consulté le : **11 mars 2025**
Description : Directives pour la conception, la normalisation et la sécurisation des APIs REST.
6. **Modern Data Stack (Fivetran Blog)**
<https://fivetran.com/blog/modern-data-stack>
Consulté le : **16 mars 2025**
Description : Analyse des tendances et outils de la data stack moderne.
7. **DataOps Manifesto**
<https://www.dataopsmanifesto.org/>
Consulté le : **20 mars 2025**
Description : Principes et bonnes pratiques pour l'industrialisation des pipelines de données.
8. **YouTube – Apache NiFi Tutorials (DataCouch)**
<https://www.youtube.com/playlist?list=PLf0swTFhTI8q18p6U6QvFqQnQKp1K0QdC>
Consulté le : **9 mars 2025**
Description : Tutoriels vidéo pour la prise en main et l'automatisation de flux avec NiFi.
9. **Towards Data Science (Medium)**
<https://towardsdatascience.com/>
Consulté le : **23 mars 2025**
Description : Articles de vulgarisation et d'approfondissement sur la data science, l'IA et l'ingénierie des données.
10. **Awesome Data Engineering (GitHub)**
<https://github.com/igorbarinov/awesome-data-engineering>

Consulté le : **27 mars 2025**

Description : Liste collaborative de ressources, outils et lectures recommandées pour l'ingénierie des données.

11. **Le Monde Informatique – Dossiers Data**

<https://www.lemondeinformatique.fr/dossiers/data/>

Consulté le : **28 février 2025**

Description : Dossiers et actualités sur la transformation digitale, la gouvernance et la valorisation des données en entreprise.

12. **Qwen (Alibaba Cloud)**

<https://qwen.alibaba.com/>

Consulté le : **4 mars 2025**

Description : Modèle d'IA générative open source développé par Alibaba Cloud, utilisé pour la génération de texte et l'assistance à la rédaction technique.

13. **Gemini (Google AI)**

<https://deepmind.google/technologies/gemini/>

Consulté le : **8 mars 2025**

Description : Modèle d'IA multimodal de Google, utilisé pour la recherche, la génération de code et l'analyse de données complexes.

14. **Google Cloud Copilot**

<https://cloud.google.com/ai/copilot>

Consulté le : **12 mars 2025**

Description : Assistant IA de Google Cloud pour l'aide à la programmation, la génération de code et l'automatisation des workflows cloud.